

การใช้ REACT ร่วมกับ REST API (1)

030513702 : Selected Topics in Software Development
Asst. Prof. Dr. Phollakrit Wongsantisuk

การใช้ Node สำหรับ Web Server

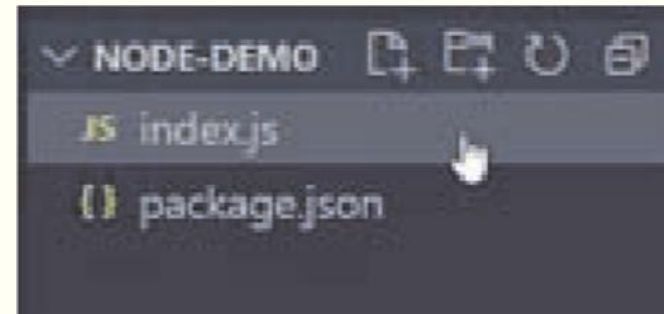
Node.js เป็นเครื่องมือในการจัดเตรียมสภาพแวดล้อมร่วมกับ Google V8 Engine สำหรับการประมวลผลคำสั่ง JavaScript ดังนั้น Node จึงไม่ใช่ภาษาคอมไพเตอร์และไม่ใช้ตัวแปลคำสั่งโดยตรง ซึ่งในการสร้าง React Application ที่ผ่านๆ มา เราจะเห็นได้ว่า มีโฟลเดอร์ชื่อ `node_modules` ถูกสร้างมาให้พร้อมกับไฟล์โมดูลที่เขียนด้วยโค้ด JavaScript ในแบบของ Node.js วัตถุประสงค์หลักของ Node.js ก็เพื่อใช้งานทางด้าน Web Server เช่นเดียวกับเทคโนโลยีอื่นๆ เช่น PHP หรือ Django เป็นต้น

เราจะเรียนรู้วิธีการนำ Node.js มาใช้งานทางฝั่งเซิร์ฟเวอร์ ร่วมกับ React ที่ทำงานทางฝั่งไคลเอนต์ ดังแนวทางต่อไปนี้

การใช้ Node สำหรับ Web Server

1. ที่ Terminal ให้ลองใช้คำสั่ง `mkdir` สร้างโฟลเดอร์เพื่อเก็บไฟล์ทางฝั่งเซิร์ฟเวอร์ (ยังไม่เกี่ยวกับ React) ในที่นี้กำหนดชื่อเป็น `node-demo`
2. เข้าไปในโฟลเดอร์ดังกล่าว แล้วติดตั้งไฟล์ด้วยคำสั่ง `npm init -y` (ใช้สำหรับสร้างไฟล์ `package.json` ใหม่ สำหรับโปรเจกต์ Node.js โดยอัตโนมัติด้วยการตั้งค่าเริ่มต้นทั้งหมดโดยไม่ต้องกรอกข้อมูลจากผู้ใช้สำหรับแต่ละตัวเลือก)
3. ใช้คำสั่ง `code . -r` เพื่อแสดงโค้ด
4. สร้างไฟล์ `index.js` เพิ่มเข้ามาในแอป
5. กำหนดโค้ดสำหรับทดสอบการทำงานของฝั่งเซิร์ฟเวอร์ลงในไฟล์ `index.js` ดังนี้

```
C:\react>mkdir node-demo  
  
C:\react>cd node-demo  
  
C:\react\node-demo>npm init -y
```



การใช้ Node สำหรับ Web Server

react/node-demo/index.js

```
const http = require('http')
const port = 8000

http.createServer((request, response) => {
  response.writeHead(200, {'Content-Type': 'text/html'})
  response.write(`
    <!doctype html>
    <html>
    <head>
      <title>Node.js</title>
    </head>
```

```
    <body>
      <h3>Welcome to Node.js</h3>
      <b>Node.js runs JavaScript on Server-Side</b>
    </body>
    </html>
  `)
  response.end()
}).listen(port)
console.log('Server listening on port '+port)
```

รันทดสอบโดยพิมพ์คำสั่ง `node index.js` ลงใน Terminal จากนั้นต้องเปิดเว็บเบราว์เซอร์ขึ้นมาเอง แล้วกำหนด URL เป็น `http://localhost:8000`

การใช้ Node สำหรับ Web Server

ถ้าต้องการหยุดรัน ก็กด <Ctrl + C> ตามเดิม หรือหากมีการแก้ไขโค้ด ก็ต้องหยุดรันแล้วค่อยรันใหม่ การเปลี่ยนแปลงจึงจะมีผล ซึ่งเราอาจลดความยุ่งยากดังกล่าว โดยการติดตั้ง Nodemon ดั่งขั้นตอนต่อไปนี้

- ที่ Terminal ให้พิมพ์คำสั่งเพื่อติดตั้ง Nodemon ดังนี้

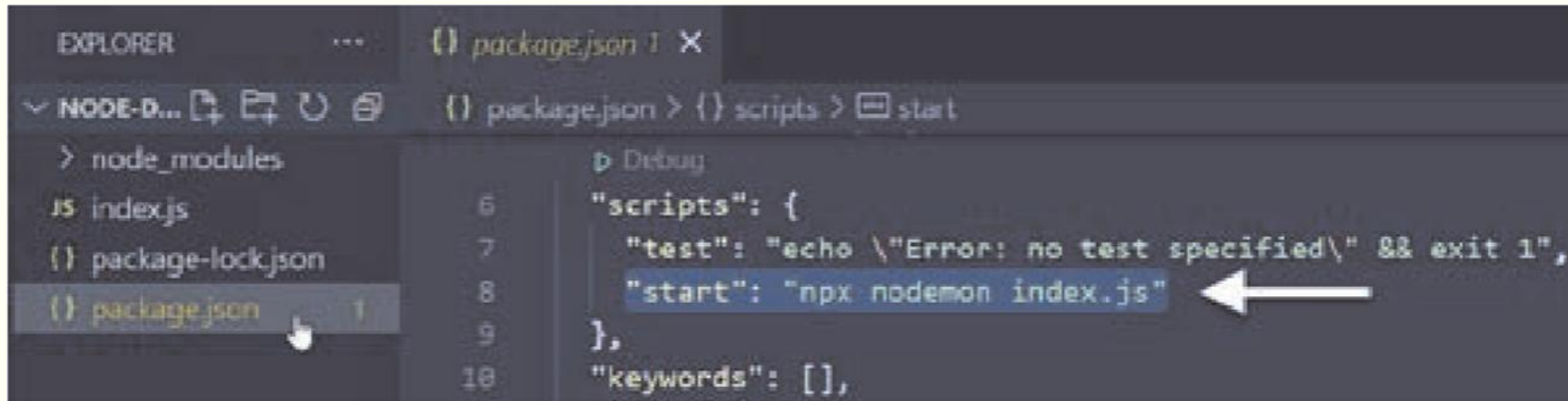
```
npm install nodemon
```

```
C:\react\node-demo>npm install nodemon
```

- ปกติการรันด้วย Nodemon เราต้องใช้คำสั่ง `npx nodemon <ชื่อไฟล์>` แต่หากเราต้องการให้เป็นแนวทางเดียวกับของ React นั่นคือ รันด้วยคำสั่ง `npm start` ก็อาจตั้งค่าที่ไฟล์ `package.json` โดยเพิ่มคำสั่งต่อไปนี้ลงในบล็อก `scripts` แล้วบันทึกการเปลี่ยนแปลง

```
"start" : "npx nodemon index.js"
```

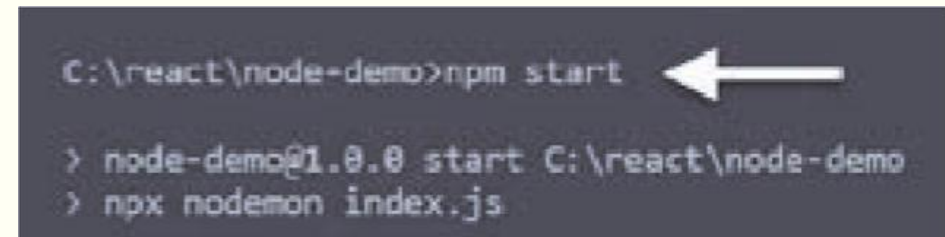
การใช้ Node สำหรับ Web Server



The screenshot shows the VS Code interface. On the left, the Explorer sidebar is open, showing a file tree with 'index.js', 'package-lock.json', and 'package.json'. 'package.json' is selected. The main editor area displays the content of 'package.json'. The 'scripts' section is visible, with the 'start' script highlighted: `"start": "npx nodemon index.js"`. A white arrow points to this line.

```
{} package.json 1 X
{} package.json > {} scripts > start
> node_modules
js index.js
{} package-lock.json
{} package.json 1
{} package.json
6 "scripts": {
7   "test": "echo \"Error: no test specified\" && exit 1",
8   "start": "npx nodemon index.js"
9 },
10 "keywords": [],
```

หมายความว่า เมื่อเราใช้คำสั่ง `npm start` ก็ให้เรียกคำสั่ง `npx nodemon index.js` เพื่อรันไฟล์ `index.js` ซึ่งต่อไปเมื่อเราแก้ไขไฟล์แล้วบันทึกการเปลี่ยนแปลง ก็ไม่จำเป็นต้องหยุดรัน เพราะการเปลี่ยนแปลงนั้นจะส่งผลทันที



The screenshot shows a terminal window with the following commands and output:

```
C:\react\node-demo>npm start
> node-demo@1.0.0 start C:\react\node-demo
> npx nodemon index.js
```

A white arrow points to the `npm start` command in the first line.

การใช้ Express สำหรับ Web Server

หากเราจัดการทางฝั่งเซิร์ฟเวอร์ด้วย Node.js เพียงอย่างเดียวตัวหัวข้อที่แล้ว อาจเกิดความยุ่งยากในหลายกรณี เช่น การกำหนดพารามิเตอร์สำหรับแต่ละ Request รวมถึงการส่งผลลัพธ์กลับไปยังฝั่งไคลเอนต์ เป็นต้น ดังนั้น โดยส่วนใหญ่นิยมใช้ร่วมกับ Express.js ซึ่งเป็น JavaScript Framework สำหรับใช้งานร่วมกับ Node.js และต่อไปเราก็จะดำเนินการทางฝั่งเซิร์ฟเวอร์ในรูปแบบของ Express แทนการใช้ Node โดยตรง สำหรับการติดตั้ง Express ให้พิมพ์คำสั่งลงบน Terminal ดังนี้

```
npm install express
```

```
C:\react\node-demo>npm install express
```

การใช้ Express สำหรับ Web Server

แล้วต่อไป เราต้องเปลี่ยนแนวการเขียนโค้ดที่ไฟล์ index.js โดยใช้รูปแบบของ Express ดังนี้

react/node-demo/index.js

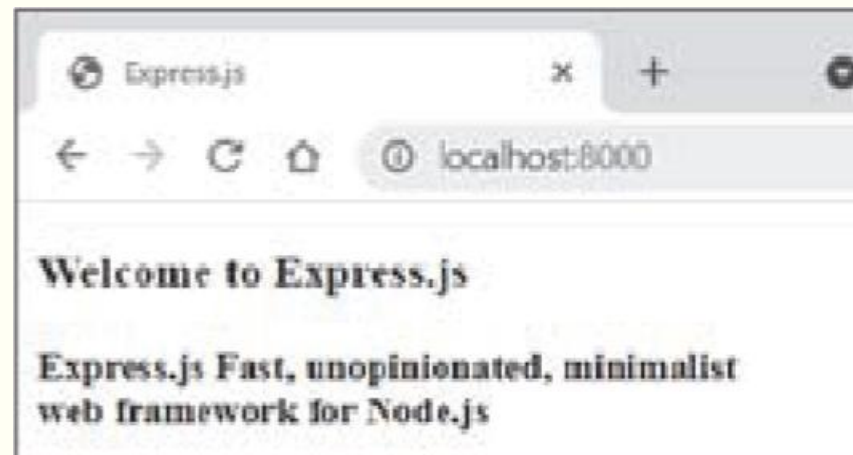
```
const express = require('express')
const app = express()
const port = 8000

app.get('/', (request, response) => {
  response.send(`
    <!doctype html>
    <html>
    <head>
      <title>Express.js</title>
    </head>
```

```
    <body>
      <h3>Welcome to Express.js</h3>
      <b>Express.js Fast, unopinionated, minimalist <br>
      web framework for Node.js</b>
    </body>
    </html>
  `)
})
app.listen(port, () => console.log('Server listening on port '+port))
```

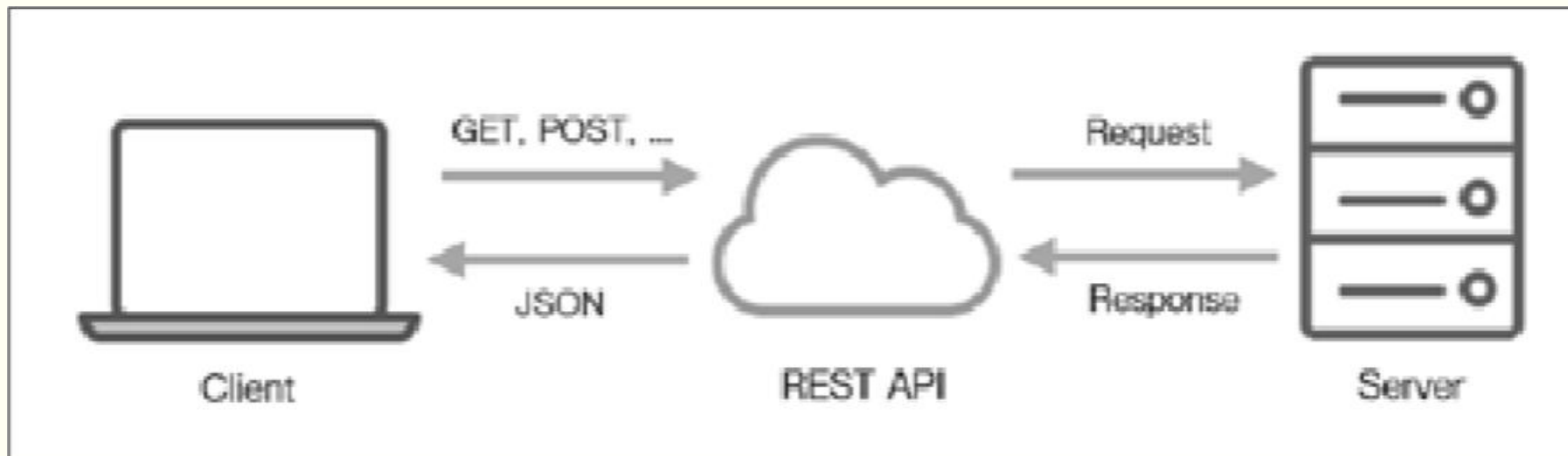
การใช้ Express สำหรับ Web Server

สำหรับการรันทดสอบก็ทำเหมือนเดิม โดยหากเราได้เซตคำสั่ง start ไว้ในไฟล์ package.json ดังหัวข้อที่ผ่านมา ก็สามารถสั่งรันด้วยคำสั่ง npm start แต่หากไม่ได้เซต ก็ต้องรันด้วย node index.js ตามเดิม ซึ่งหลังจากสั่งรัน เราก็กำหนด URL ให้กับเบราว์เซอร์เป็น <http://localhost:8000> เช่นเคย



การใช้ React ร่วมกับ Node และ Express

การใช้ React ที่เราได้เรียนรู้ในบทที่ผ่านมา มาทั้งหมดนั้น เป็นการใช้งานทางฝั่งไคลเอนต์ (Local) หรือ Frontend เพียงอย่างเดียว และในลำดับต่อไป เราจะนำแอปทางฝั่งไคลเอนต์ ไปเชื่อมต่อกับทางฝั่งเซิร์ฟเวอร์ (Server) หรือ Backend โดยใช้รูปแบบที่เรียกว่า REST API (REST: Representational State Transfer) ซึ่งเป็นการรับส่งข้อมูลผ่านโปรโตคอล HTTP แบบเบื่องหลังคล้ายกับ AJAX และโดยทั่วไปเราถือว่า REST API ไม่ใช่สิ่งเดียวกับ AJAX เพราะความแตกต่างของเทคโนโลยีที่ใช้สร้างมันขึ้นมา แต่หลักการต่างๆ ที่ใช้ดำเนินการก็แทบจะไม่ต่างกัน ดังแนวทางในภาพต่อไปนี้



การใช้ React ร่วมกับ Node และ Express

การเชื่อมโยงระหว่างฝั่งไคลเอนต์ (หรือ Client) และเซิร์ฟเวอร์ จะทำให้แอปสามารถทำงานได้ครบสมบูรณ์ตามหลักการใช้งานจริงของ Web Application อย่างไรก็ตามถึงแม้จะแบ่งออกเป็น 2 ฝั่ง แต่เราสามารถสร้างองค์ประกอบทั้งหมด ให้รวมอยู่ภายในแอปเดียวกัน ซึ่งก็เทียบได้กับไฟล์ทั้งหมดของเว็บไซต์นั่นเอง โดยจะกำหนดโครงสร้างของแอปให้มีลักษณะดังภาพ

- **โฟลเดอร์ local** (จะกำหนดเป็นชื่ออื่นๆ ก็ได้ เช่น client, frontend, react เป็นต้น) ใช้สำหรับจัดเก็บไฟล์ที่ทำงานทางฝั่งไคลเอนต์ ซึ่งในกรณีที่เรากำลังศึกษาอยู่นี้ก็คือ React Application นั่นเอง
- **โฟลเดอร์ server** (จะกำหนดเป็นชื่ออื่นๆ ก็ได้ เช่น api, backend เป็นต้น) สำหรับจัดเก็บไฟล์ที่ทำงานทางฝั่งเซิร์ฟเวอร์ (Node/Express)

สำหรับการติดตั้งไฟล์ทั้งสองฝั่ง ขอนำมากล่าวในภาพรวมพอสังเขปอีกครั้ง รวมถึงการตั้งค่าเพิ่มเติมในบางขั้นตอนที่ยังไม่ได้กล่าวมาถึงมาก่อน ดังต่อไปนี้

การติดตั้งไฟล์ที่ฝั่ง Local (React App)

สำหรับฝั่งโลคอล ในที่นี้คือ React Application ส่วนใหญ่ก็ใช้หลักการเดิม และก็มี การตั้งค่าเพิ่มเติมอีกเล็กน้อย
ดังนี้

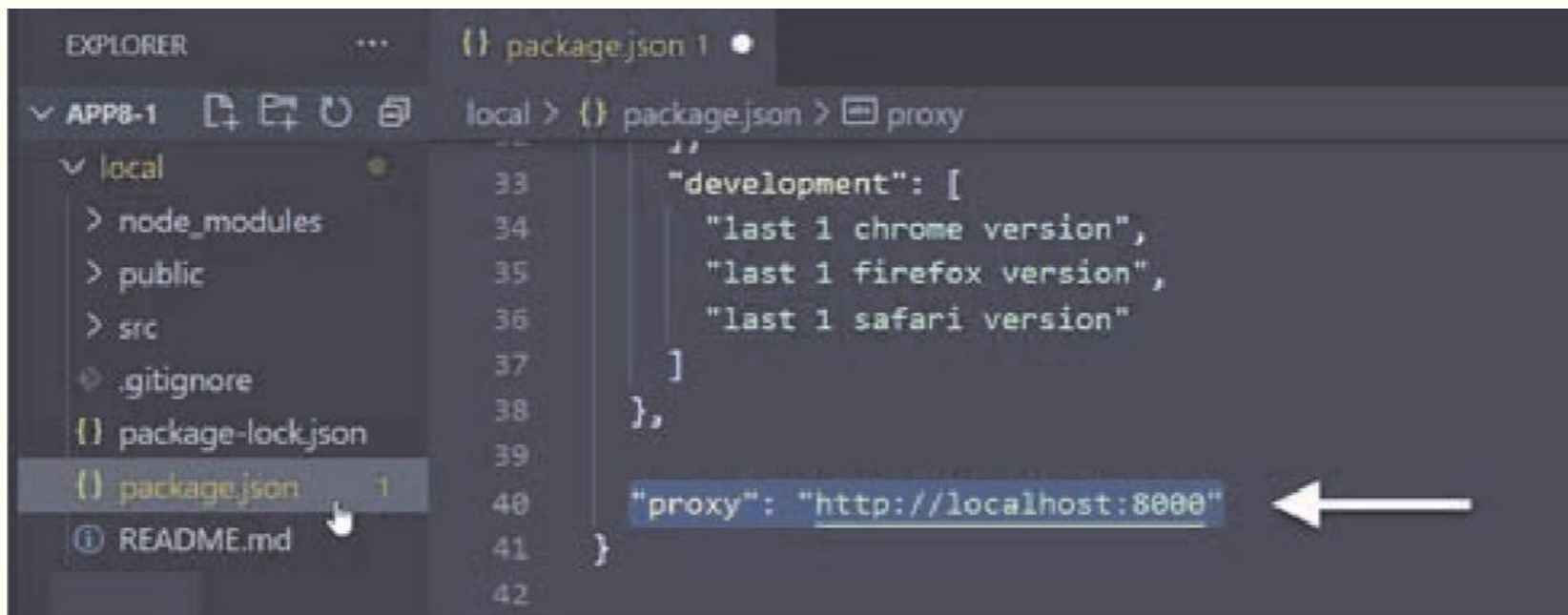
1. เริ่มจากการสร้างโฟลเดอร์รวมของแอปขึ้นมาก่อน เพื่อจะใช้เก็บไฟล์ทั้งโลคอลและเซิร์ฟเวอร์ ซึ่งเพื่อให้สอดคล้องกับบทอื่นๆ ในที่นี้ขอกำหนดชื่อเป็น app8-1
2. เข้าไปในโฟลเดอร์ app8-1 แล้วใช้คำสั่ง create-react-app เพื่อติดตั้งไฟล์ลงในโฟลเดอร์ที่ชื่อ local ดังภาพ

```
C:\react>mkdir app8-1  
  
C:\react>cd app8-1  
  
C:\react\app8-1>create-react-app local
```

การติดตั้งไฟล์ที่ฝั่ง Local (React App)

3. ตามปกติ เมื่อฝั่งโลคอลติดต่อกับฝั่งเซิร์ฟเวอร์ จะต้องระบุปลายทางเป็น URL แบบเต็มๆ เช่น <http://localhost:8000/api/test/example> ซึ่งดูยืดยาวเกินไป หากเราต้องการให้มันกระชับขึ้นโดยระบุแค่ส่วน path เช่น /api/test/example ก็อาจเซตค่า proxy โดยเปิดไฟล์ package.json ทางฝั่งโลคอลแล้วเพิ่มคำสั่งดังนี้

```
"proxy" : "http://localhost:8000"
```



The screenshot shows the VS Code interface with the Explorer on the left and the Editor on the right. The Explorer shows a project named 'APP8-1' with a folder 'local' containing files like 'node_modules', 'public', 'src', '.gitignore', 'package-lock.json', 'package.json', and 'README.md'. The 'package.json' file is selected and open in the Editor. The Editor shows the 'development' section of the 'scripts' object, which includes 'last 1 chrome version', 'last 1 firefox version', and 'last 1 safari version'. Below this, the 'proxy' property is set to 'http://localhost:8000', which is highlighted with a blue selection bar and a white arrow pointing to it from the right.

```
33  "development": [  
34    "last 1 chrome version",  
35    "last 1 firefox version",  
36    "last 1 safari version"  
37  ],  
38  
39  
40  "proxy": "http://localhost:8000"  
41  
42  }
```

การติดตั้งไฟล์ที่ฝั่ง Server (Node/Express)

สำหรับการติดตั้งทางฝั่งเซิร์ฟเวอร์ สามารถสรุปขั้นตอนได้ดังนี้

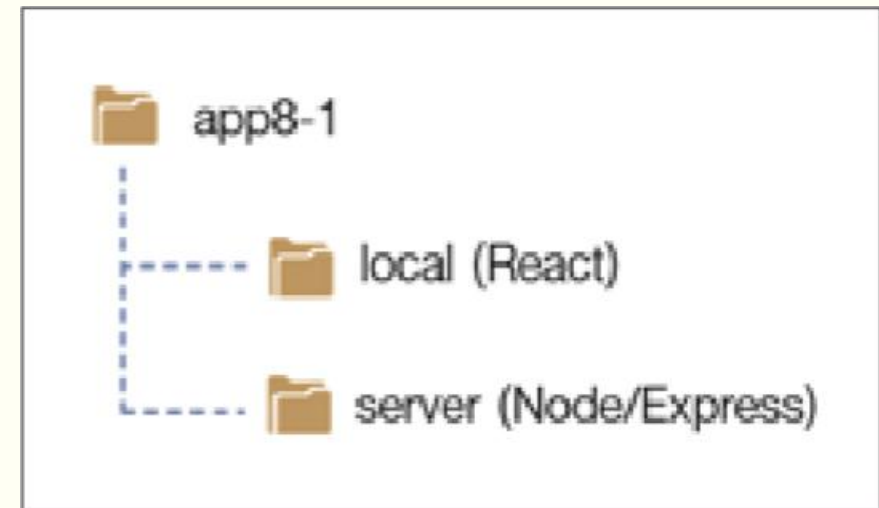
1. ให้สร้างโฟลเดอร์สำหรับจัดเก็บไฟล์ทางฝั่งเซิร์ฟเวอร์ โดยโฟลเดอร์ดังกล่าว ควรให้อยู่ในระดับเดียวกับโฟลเดอร์ local ในที่นี้ตั้งชื่อโฟลเดอร์เป็น server
2. เข้าไปในโฟลเดอร์ server แล้วพิมพ์คำสั่ง `npm init -y`
3. ติดตั้งไลบรารีที่จำเป็นลงโฟลเดอร์ server เช่น `express`, `nodemon` เป็นต้น

```
c:\react\app8-1>mkdir server
```

```
c:\react\app8-1>cd server
```

```
c:\react\app8-1\server>npm init -y
```

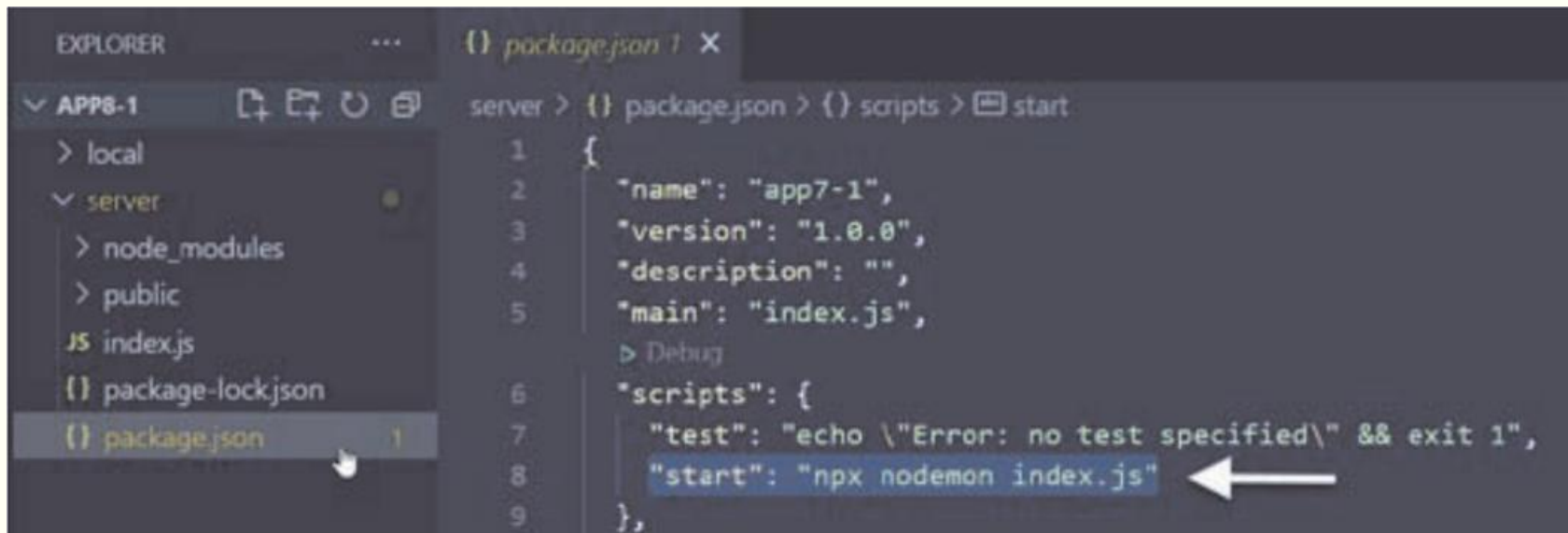
```
c:\react\app8-1\server>npm install express nodemon
```



4. สร้างไฟล์ `index.js` เพิ่มลงในโฟลเดอร์ server

การติดตั้งไฟล์ที่ฝั่ง Server (Node/Express)

5. เพื่อให้เราสามารถรันฝั่งเซิร์ฟเวอร์ด้วยคำสั่ง `npm start` ได้เช่นเดียวกับ React ให้เปิดไฟล์ `server/package.json` แล้วเพิ่มคำสั่ง `start` ลงในบล็อก `script` ดังนี้ (กรณีนี้ ต้องติดตั้ง Nodemon เอาไว้แล้ว)



```
server > {} package.json > {} scripts > start
1  {
2    "name": "app7-1",
3    "version": "1.0.0",
4    "description": "",
5    "main": "index.js",
6    "scripts": {
7      "test": "echo \"Error: no test specified\" && exit 1",
8      "start": "npx nodemon index.js"
9    },
```

จากขั้นตอนที่ผ่านมาเป็นพื้นฐานการติดตั้งไฟล์ทั้ง 2 ฝั่ง และต่างก็มีโฟลเดอร์ `node_modules` ด้วยกันทั้งคู่ ดังนั้น หากเราจะติดตั้งไลบรารี (โมดูล) อื่นๆ เพิ่มเติม ต้องพิจารณาก่อนว่า โมดูลดังกล่าวจะใช้ทางฝั่งใด ถ้าใช้ร่วมกับ React เช่น `bootstrap`, `react-router-dom` ก็ให้ติดตั้งทางฝั่งไคลเอนต์ นั่นคือ ต้องเข้าไปในโฟลเดอร์ `local` ก่อน แล้วค่อยติดตั้งด้วย `npm` แต่หากเป็นโมดูลที่ต้องใช้ร่วมกับ Node/Express ก็ให้ติดตั้งทางฝั่งเซิร์ฟเวอร์ โดยเข้าไปในโฟลเดอร์ `server` ก่อน แล้วค่อยติดตั้งด้วย `npm`

การรันทดสอบฝั่ง Local และ Server

หากเราจะทดสอบการเชื่อมต่อระหว่างฝั่งโลคอลและเซิร์ฟเวอร์ จะต้องรันทั้ง 2 ฝั่งพร้อมๆ กัน ดังแนวทางต่อไปนี้

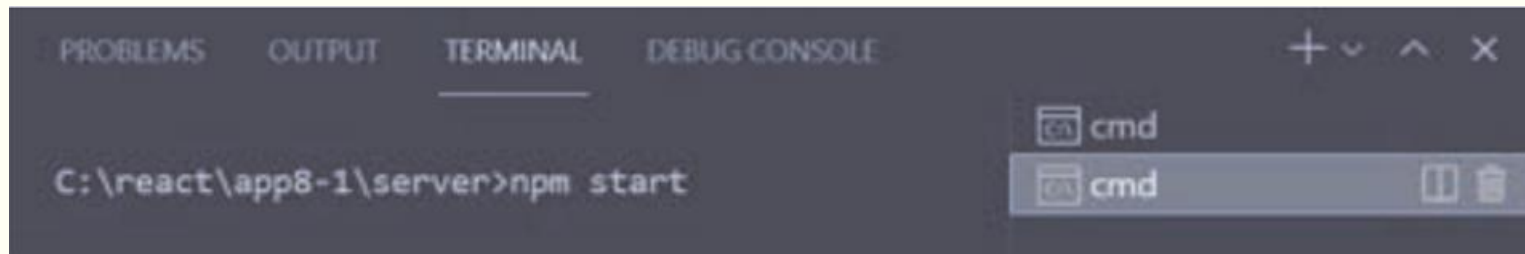
- **การรันฝั่งโลคอล** ซึ่งในที่นี้ก็คือ React และเราจัดเก็บไฟล์ไว้ในโฟลเดอร์ local ดังนั้น เราต้องเข้าไปในโฟลเดอร์ดังกล่าว แล้วรันด้วยคำสั่ง **npm start** ตามปกติ



The image shows a screenshot of a Visual Studio Code terminal window. The terminal has tabs for 'PROBLEMS', 'OUTPUT', 'TERMINAL', and 'DEBUG CONSOLE'. The 'TERMINAL' tab is active. The command prompt shows the current directory as 'C:\react\app8-1\local' and the command 'npm start' has been entered. On the right side of the terminal window, there is a sidebar with two 'cmd' icons, each with a window icon and a trash icon.

การรันทดสอบฝั่ง Local และ Server

- การรันฝั่งเซิร์ฟเวอร์ ให้ทำตามขั้นตอนดังนี้
 - หากใช้ Terminal รันฝั่งโลคอลไปแล้ว ให้เพิ่ม Terminal ใหม่เข้ามาอีกอัน เพื่อรันฝั่งเซิร์ฟเวอร์โดยคลิก ไอคอน + ตรงแถบบริเวณมุมขวาบนของ Terminal หรือเลือกที่เมนู **Terminal > New Terminal**
 - ที่ Terminal ใหม่ที่เพิ่มเข้ามา ให้เราเข้าไปในโฟลเดอร์ server และหากเราเซตคำสั่ง start ในไฟล์ server/package.json ดังที่ได้กล่าวไปแล้ว ก็รันด้วยคำสั่ง npm start เช่นเดียวกัน



หากต้องการหยุดรันก็เปิด Terminal ของฝั่งนั้นๆ แล้วกด <Ctrl + C> เหมือนเดิม สำหรับทางฝั่งเซิร์ฟเวอร์นั้น ถ้าเราติดตั้ง Nodemon และเซตค่าเอาไว้แล้ว หากแก้ไขโค้ดและบันทึก การเปลี่ยนแปลงก็จะส่งผลทันที โดยไม่ต้องหยุดรัน

การใช้ REST API ด้วยฟังก์ชัน fetch()

การรับส่งข้อมูลระหว่างฝั่งไคลเอนต์และเซิร์ฟเวอร์ จะดำเนินการในรูปแบบของ REST API ซึ่งก็คล้ายกับ AJAX ที่บางคนอาจเคยใช้กันมาแล้ว โดยขั้นตอนทางฝั่งไคลเอนต์ มีหลายทางเลือกที่เรานำมาใช้ได้ แต่ที่ได้รับความนิยมในปัจจุบันก็คือ ฟังก์ชัน fetch() ของ JavaScript ดังรายละเอียดต่อไปนี้

รูปแบบของฟังก์ชัน fetch() ทางฝั่ง Local

สำหรับฟังก์ชัน fetch() หรือ window.fetch() เป็นองค์ประกอบที่มีอยู่แล้วใน JavaScript ดังนั้น เราจึงไม่จำเป็นต้องติดตั้งสิ่งใดเพิ่มเติมอีก แต่สามารถใช้งานได้ทันที โดยรูปแบบพื้นฐานของฟังก์ชัน fetch() คือ

```
fetch('url', {  
  method: 'GET' | 'POST' | 'PUT' | 'DELETE',  
  body: ข้อมูลที่จะส่งไปยังเซิร์ฟเวอร์, // สำหรับกรณี method: 'POST'  
  headers: { ... }  
})  
.then(response => จัดการข้อมูลผลลัพธ์ที่ส่งกลับมา)  
.then(result => นำผลลัพธ์ที่ได้ไปใช้งานตามต้องการ)  
.catch(error => ดำเนินการกับข้อผิดพลาด)
```

การใช้ REST API ด้วยฟังก์ชัน `fetch()`

- url ให้ระบุพาธปลายทางที่ฝั่งเซิร์ฟเวอร์ ซึ่งเราจะส่ง Request ขึ้นไป ทั้งนี้ หากเราไม่ได้ตั้งค่า proxy ที่ไฟล์ `package.json` ก็ต้องระบุ URL แบบเต็ม เช่น `http://localhost:8000/path/` แต่หากเราตั้งค่า proxy เอาไว้แล้ว ให้ระบุแค่ส่วนพาธของมัน และโดยทั่วไปเรานิยมให้ชื่อพาธทางฝั่งเซิร์ฟเวอร์ขึ้นต้นด้วย `/api` เพื่อบ่งชี้ว่า เป็นการสื่อสารแบบ REST API เช่น `/api/search` หรือ `/api/member/signin` เป็นต้น แต่ไม่ใช่ข้อบังคับที่จำเป็น เราอาจเลือกใช้คำอื่นก็ได้ เช่น `/server` เป็นต้น
- method สำหรับกำหนดเมธอดที่จะส่งข้อมูลขึ้นไป ซึ่งมีให้เลือกหลายอย่าง เช่น GET, POST, PUT, DELETE เป็นต้น แต่โดยส่วนใหญ่ เรามักเลือกแบบ GET สำหรับข้อมูลสั้นๆ ที่สามารถแนบต่อท้าย URL ได้ หรือใช้ POST สำหรับข้อมูลที่ยาวๆ และต้องการความปลอดภัยในการจัดส่ง
- body สำหรับกำหนดข้อมูลที่จะส่งขึ้นไปเมื่อใช้เมธอด POST โดยข้อมูลดังกล่าวต้องแปลงให้อยู่ในรูปแบบ JSON
- headers โดยทั่วไปจะกำหนด Content-Type เมื่อส่งข้อมูลขึ้นไปในแบบ JSON
- `then()` อันแรก ใช้ในการรับข้อมูลที่ตอบสนอง (Response) จากทางฝั่งเซิร์ฟเวอร์ตามรูปแบบของมัน ทั้งนี้หากข้อมูลถูกส่งกลับมาแบบ JSON ก็ต้องแปลงด้วยเมธอด `json()` หรือถ้าเป็นข้อความก็ต้องแปลงด้วยเมธอด `text()` แล้วหลังจากนั้น มันก็จะถูกส่งต่อไปให้กับ `then()` อันที่สองโดยอัตโนมัติ

การใช้ REST API ด้วยฟังก์ชัน fetch()

- then() อันที่สอง จะรับผลลัพธ์ที่ส่งมาจาก then() อันแรก แล้วจากนั้นเราก็นำผลลัพธ์ดังกล่าวไปดำเนินการตามต้องการ เช่น แสดงผล
- catch() หากเราคาดว่า อาจเกิดข้อผิดพลาดเกิดขึ้นในระหว่างการรับส่งข้อมูล ก็อาจเพิ่มบล็อก catch เพื่อจัดการกับข้อผิดพลาดดังกล่าว

กรณีที่เราจะส่งข้อมูลด้วยเมธอด GET จะต้องแนบต่อท้าย URL ในแบบ Query String หรือ Search แต่จะกำหนดให้แก่ออปชัน body ไม่ได้ โดยข้อมูลดังกล่าวอาจกำหนดขึ้นมาเองในแบบสตริง หรือจะสร้างด้วย URLSearchParams ก็ได้ ดังแนวทางต่อไปนี้

การใช้ REST API ด้วยฟังก์ชัน fetch()

```
const params = new URLSearchParams()
params.append('keyword', 'react')
params.append('page', 1)
const url = '/api/search?' + params
/*ลักษณะ url ที่ได้คือ
/api/serach?keyword=react&page=1
*/

fetch(url, {
  method: 'GET'
})
.then(response => {
  return response.json()           //คืนค่าไปยัง then() อันที่ 2
})
.then(result => {                   //ผลลัพธ์จาก then() อันแรก
  alert(result.data)               //เทคนิคการแสดงผลจะกล่าวถึงในหัวข้อต่อไป
})
.catch(err => {
  alert(err)
})
```

การใช้ REST API ด้วยฟังก์ชัน fetch()

เนื่องจากเมธอด GET เป็นค่าดีฟอลต์อยู่แล้ว ดังนั้น ในกรณีที่ไมระบุเมธอด จะถือว่าใช้แบบ GET และตามหลักการของ Arrow Function เราสามารถให้กระชับขึ้นได้ดังนี้

```
...  
fetch(url) //ไม่ระบุเมธอด ถือว่าใช้แบบ GET  
.then(response => response.json())  
.then(result => alert(result.data))  
.catch(err => alert(err))
```

ส่วนการส่งข้อมูลด้วยเมธอด POST นั้น เราต้องกำหนดข้อมูลในแบบออบเจ็กต์ จากนั้นแปลงเป็น JSON พร้อมกำหนด header ให้ฝั่งเซิร์ฟเวอร์รับทราบและเลือกวิธีการได้ถูกต้อง ดังแนวทางต่อไปนี้

```
const data = {  
  login: 'someone',  
  password: 'qwerty'  
}  
  
fetch('/api/member/signin', {  
  method: 'POST',  
  body: JSON.stringify(data),
```

การใช้ REST API ด้วยฟังก์ชัน fetch()

```
        headers: {'Content-Type': 'application/json'}  
    ))  
    .then(response => response.text())  
    .then(result => alert(result))  
    .catch(err => alert(err))
```

สำหรับแนวทางการนำไปใช้งาน ให้ดูร่วมกับหัวข้อต่อไป

การใช้ REST API ด้วยฟังก์ชัน `fetch()`

การใช้ `fetch()` ร่วมกับ Function Component

ตามหลักการของ React ข้อมูลผลลัพธ์ที่ถูกส่งกลับมาจากเซิร์ฟเวอร์ ตามปกติเราจะนำไปแสดงผลในคอมโพเนนต์ ดังนั้น จุดที่เราจะทำการสื่อสารผ่าน REST API เพื่อรับส่งข้อมูลระหว่างฝั่งไคลเอนต์และเซิร์ฟเวอร์ก็ควรจะอยู่ในคอมโพเนนต์อันนั้นด้วย และสำหรับในกรณีของ Function Component หากเราต้องการเชื่อมต่อกับเซิร์ฟเวอร์ทันทีหลังจากแสดงผลคอมโพเนนต์ ก็อาจเรียก `fetch()` ภายในเมธอด `useEffect()` แต่อาจมีคำแจ้งเตือนปรากฏบน Terminal ซึ่งเราสามารถปิดคำแจ้งเตือนนี้ได้โดยเพิ่มคอมเมนต์

```
/* eslint-disable react-hooks/exhaustive-deps */  
ไว้ที่ส่วนบนสุดของไฟล์ที่ใช้สร้างคอมโพเนนต์นั้น ดังโค้ดต่อไปนี้
```

การใช้ REST API ด้วยฟังก์ชัน fetch()

```
/* eslint-disable react-hooks/exhaustive-deps */
import ...

function ProductDetail() {
  React.useEffect(() => {
    fetch('/api/products/detail?id=100')
      .then(response => response.text())
      .then(result => alert(result))
      .catch(err => alert(err))
  }, [])

  return <div></div>
}
```

การใช้ REST API ด้วยฟังก์ชัน fetch()

การใช้ fetch() ร่วมกับ Class Component

ในกรณีของ Class Component หากเราต้องการเชื่อมต่อกับเซิร์ฟเวอร์ทันทีหลังจากแสดงผลคอมโพเนนต์ให้เรา กำหนดฟังก์ชัน fetch() ไว้ในเมธอด componentDidMount() ซึ่งตามหลักการที่เราได้ศึกษามาแล้ว เมธอดนี้จะถูกเรียกขึ้นมาทำงานต่อจาก render() ดังแนวทางต่อไปนี้

```
class ProductDetail extends React.Component {  
  componentDidMount {  
    fetch('/api/products/detail?id=100')  
      .then(response => response.text())  
      .then(result => alert(result))  
      .catch(err => alert(err))  
  }  
  
  render() {  
    return <div></div>  
  }  
}
```

การใช้ REST API ด้วยฟังก์ชัน fetch()

การใช้ fetch() ร่วมกับ Event Handler

หากเราต้องการเชื่อมต่อกับเซิร์ฟเวอร์ในขณะใดขณะหนึ่ง ซึ่งตามปกติก็คือ เมื่อเกิดอีเวนต์นั่นเอง เช่น เมื่อคลิกปุ่ม Submit ก็ให้อ่านข้อมูลจากฟอร์มแล้วส่งผ่าน REST API ไปยังเซิร์ฟเวอร์ เป็นต้น สำหรับกรณีนี้ เราเพียงแค่งำหนดฟังก์ชัน fetch() ไว้ใน Event Handler ที่ใช้จัดการอีเวนต์ ส่วนหลักการทำงานอื่นๆ ก็เหมือนเดิม เช่น

```
function ProductDetail() {  
  const onClickButton = () => {  
    fetch('/api/products/detail?id=100')  
      .then(response => response.text())  
      .then(result => alert(result))  
      .catch(err => alert(err))  
  }  
  
  return <button onClick={onClickButton}>Show Detail</button>  
}
```

การใช้ REST API ด้วยฟังก์ชัน `fetch()`

ข้อกำหนดพื้นฐานทางฝั่ง Server

ทางฝั่งเซิร์ฟเวอร์นั้น เราจะใช้ไฟล์หลักคือ `index.js` เป็นศูนย์กลางการรับส่งข้อมูล โดยลักษณะสำคัญเมื่อเราใช้งานร่วมกับ REST API ก็คือ

- เพื่อให้ฝั่งเซิร์ฟเวอร์ (Express) รองรับการอ่านข้อมูลที่ส่งมาจากฝั่งไคลเอนต์ได้ จะต้องเพิ่มโค้ดลงใน `index.js` ดังนี้

```
const express = require('express')
const app = express()

//express 4.16.0+
app.use(express.urlencoded({extended: true}))
app.use(express.json())
...
```

การใช้ REST API ด้วยฟังก์ชัน `fetch()`

- การเลือกเมธอดและการกำหนดพารามิเตอร์ ต้องสอดคล้องกับฟังก์ชันของ REST API ดังนี้
 - หากฟังก์ชันของ REST API ส่งข้อมูลขึ้นมาด้วยเมธอด GET ทางฝั่งเซิร์ฟเวอร์ก็ต้องกำหนดพารามิเตอร์ด้วยเมธอด `app.get()` แล้วอ่านข้อมูลด้วย

`request.query.ชื่อพารามิเตอร์`

```
...
const app = express()

// ถ้าฟังก์ชันของ REST API ส่งข้อมูลแบบ GET
app.get('/api/test', (request, response) => {
  let keyword = request.query.keyword
  ...
})
```

การใช้ REST API ด้วยฟังก์ชัน `fetch()`

- ถ้าฝั่งไคลเอนต์ใช้เมธอด `POST` ฝั่งเซิร์ฟเวอร์ก็ต้องกำหนดพารามิเตอร์ด้วยเมธอด `app.post()` แล้วอ่านข้อมูลด้วย

```
request.body.ชื่ออินพุต
```

```
//ถ้าฝั่งไคลเอนต์ส่งข้อมูลแบบ POST
app.post('/api/test', (request, response) => {
  let login = request.body.login
  ...
})
```

การใช้ REST API ด้วยฟังก์ชัน fetch()

- การส่งข้อมูลกลับจากฝั่งเซิร์ฟเวอร์ไปยังไคลเอนต์ อาจเลือกแบบใดแบบหนึ่งดังนี้
 - `response.send()` เมื่อต้องการส่งแบบสตริงธรรมดา ซึ่งอาจมี HTML รวมอยู่ด้วยก็ได้ และฝั่งไคลเอนต์ก็จัดการที่ `then()` อันแรกด้วยเมธอด `response.text()`

```
//server
app.get('/api/test', (request, response) => {
  response.send('Hi React')
})
```

```
//local
fetch('/api/test')
.then(response => response.text())
.then(result => alert(result))
```

การใช้ REST API ด้วยฟังก์ชัน fetch()

- `response.json()` เมื่อต้องการส่งข้อมูลในลักษณะของอาร์เรย์ หรือ key/value เช่น { product: 'notebook', price: '30000' } และฟังก์ชันที่จัดการที่ `then()` อันแรกด้วยเมธอด `response.json()` ส่วน `then()` อันที่สอง หากต้องการเข้าถึงข้อมูลใด ก็ระบุด้วยชื่อคีย์ เช่น `result.price`

```
//server
app.get('/api/test', (request, response) => {
  response.json({product: 'notebook', price: '30000'})
})
```

```
//local
fetch('/api/test')
.then(response => response.json())
.then(result => alert(`${result.product} ราคา ${result.price}`))
```

การแสดงผลลัพธ์บนคอมพิวเตอร์ทางฝั่ง Local

การใช้ React ร่วมกับ REST API นั้น ตามปกติจะเป็นการนำข้อมูลจากฝั่งเซิร์ฟเวอร์ มาอัปเดตเนื้อหาบางส่วน ของเพจ (หรือของอีลิเมนต์อันใดอันหนึ่ง) ทางฝั่งไคลเอนต์ เนื่องจากการส่งข้อมูลผ่านเครือข่ายจะต้องใช้เวลา ดังนั้น เราอาจ แสดงเนื้อหาบางส่วนเอาไว้ล่วงหน้า โดยอีลิเมนต์ที่ต้องรอข้อมูลจากเซิร์ฟเวอร์ก็ให้แสดงเนื้อหาว่างๆ ไปก่อน แล้วเมื่อผลลัพธ์ ถูกส่งกลับมา ก็ค่อยนำไปอัปเดตให้กับอีลิเมนต์ดังกล่าว

ตามแนวทางของ React นั้น วิธีที่เหมาะสมที่สุดและสามารถนำไปใช้ได้ในทุกกรณีก็คือ การใช้ State เป็นตัว ติดตามการเปลี่ยนแปลงและอัปเดตเนื้อหาของอีลิเมนต์ที่ใช้แสดงผลลัพธ์จากเซิร์ฟเวอร์ โดยมีหลักการดังต่อไปนี้

- เราต้องสร้าง State ตามวิธีการของ Function Component หรือ Class Component เพื่อจัดเก็บผลลัพธ์ที่ส่งมา จากเซิร์ฟเวอร์
- ที่อีลิเมนต์เป้าหมาย เราต้องนำค่าจากตัวแปร State ไปกำหนดเป็นเนื้อหาของมัน
- ในส่วนของฟังก์ชัน `fetch()` เมื่อผลลัพธ์ถูกส่งกลับมาที่ `then()` เราต้องนำมาอัปเดตค่าของ State ซึ่งจะส่งผลให้ เนื้อหาของอีลิเมนต์เป้าหมายที่เชื่อมโยงอยู่กับ State ถูกอัปเดตตามไปด้วย (ความจริงมันก็คือ การ re-render หรือแสดงผลเนื้อหาของคอมพิวเตอร์อันนั้นใหม่อีกครั้ง)

การแสดงผลลัพธ์บนคอมโพเนนต์ทางฝั่ง **Local**

```
//local
...
let [data, setData] = React.useState('')
...
fetch('/api/test')
.then(response => response.text())
.then(result => setData(result)) //เมื่อผลลัพธ์ถูกส่งมา ก็อัปเดตค่าของ State
...
return <div>{data}</div>           //แสดงเนื้อหาที่จัดเก็บใน State
                                   //ถ้าค่าของ State เปลี่ยน เนื้อหา ก็จะเปลี่ยนตาม
```

การแสดงผลลัพธ์บนคอมโพเนนต์ทางฝั่ง Local

กรณีที่แสดงในโค้ดก่อนหน้านี้ เป็นการสร้าง State สำหรับ Function Component ส่วนในกรณีของ Class Component เราก็กำหนดค่าของ State เป็นแบบออบเจกต์แล้วอัปเดตด้วย `this.setState()` อย่างไรก็ตาม ในการใช้งานจริงนั้น ข้อมูลผลลัพธ์ที่กลับมาจากเซิร์ฟเวอร์ เราอาจต้องนำมาจัดรูปแบบเพิ่มเติมในแบบ JSX จากนั้นค่อยนำไปอัปเดตค่าของ State แต่กรณีที่ผลลัพธ์ถูกส่งกลับมาในแบบ text และมีแท็ก HTML รวมอยู่ด้วย หากใช้วิธีการดั่งโค้ดที่ผ่านมา แท็กจะแสดงในแบบข้อความทั่วไป ซึ่งหากเราต้องการให้แท็กเหล่านั้นถูกประมวลผลไปด้วย ก็ควรกำหนดเนื้อหาให้แก่อีลิเมนต์ด้วยแอตทริบิวต์ `dangerouslySetInnerHTML` เช่น

```
//server
app.get('/api/test', (request, response) => {
  reponse.send('<b><u>Hi React</u></b>')
})
```

```
//local
...
let [data, setData] = React.useState('')
...
fetch('/api/test')
  .then(response => response.text())
```

การแสดงผลลัพธ์บนคอมพิวเตอร์ทางฝั่ง Local

```
.then(result => setData(result))  
...  
return <div dangerouslySetInnerHTML={{__html: data}}></div>
```

สำหรับแนวทางการใช้งานโดยรวม ให้ดูจากการทดสอบต่อไปนี้ ซึ่งเมื่อคลิกปุ่มอ่านค่าเวลา ก็จะส่ง Request ออกไปด้วยฟังก์ชัน fetch() แล้วทางฝั่งเซิร์ฟเวอร์ก็จะส่งค่า ชั่วโมง:นาฬิกา:วินาที กลับคืนมาในรูปแบบ JSON จากนั้นเราก็นำไปอัปเดตค่า State แล้วอีกปุ่มหนึ่งจะเป็นการอ่านข้อมูลผลการแข่งขันจากฝั่งเซิร์ฟเวอร์มาแสดง โดยจะส่งค่ากลับมาในรูปแบบ text ที่มีแท็ก HTML รวมอยู่ด้วย ดังนั้น ฝั่งไคลเอนต์ต้องแสดงผลด้วย dangerouslySetInnerHTML

การแสดงผลลัพธ์บนคอมพิวเตอร์ทางฝั่ง Local

react/app8-1/server/index.js

```
const express = require('express')
const app = express()
const port = 8000
app.use(express.urlencoded({extended: true}))
app.use(express.json())
app.use(express.static('public'))

app.get('/api/server-time', (request, response) => {
  let now = new Date()
  let time = {
    hour: now.getHours(),
    minute: now.getMinutes(),
    second: now.getSeconds()
  }
  response.json(time)
})
```

การแสดงผลลัพธ์บนคอมพิวเตอร์ทางฝั่ง Local

react/app8-1/server/index.js (ต่อ)

```
function rd(min, max) {  
  let x = (max - min) + 1  
  return min + Math.floor(Math.random() * x)  
}
```

```
app.get('/api/football-result', (request, response) => {  
  let table = `  
    <table border="1" style="margin: 7px auto">  
      <tr><td>ManU</td><td>${rd(0, 5)}-${rd(0, 5)}</td><td>Liverpool</td></tr>  
      <tr><td>Chelsea</td><td>${rd(0, 5)}-${rd(0, 5)}</td><td>ManCity</td></tr>  
      <tr><td>Arsenal</td><td>${rd(0, 5)}-${rd(0, 5)}</td><td>Spur</td></tr>  
    </table>  
  `;  
  response.send(table)  
})
```

```
app.listen(port, () => {  
  console.log('Server listening on port ' + port)  
})
```

การแสดงผลลัพธ์บนคอมพิวเตอร์ทางฝั่ง Local

react/app8-1/local/src/rest-basic.js

```
import React from 'react'

export default function RestBasic() {
  let [serverTime, setServerTime] = React.useState("")
  let [footballResult, setFootballResult] = React.useState("")

  const onClickShowTime = () => {
    fetch('/api/server-time')
      .then(response => response.json())
      .then(result => {
        let r = <>{result.hour}:{result.minute}:{result.second}</>
        setServerTime(r)
      })
      .catch(err => alert(err))
  }
}
```

การแสดงผลลัพธ์บนคอมพิวเตอร์ทางฝั่ง Local

react/app8-1/local/src/rest-basic.js (ต่อ)

```
const onClickFootballResult = () => {
  fetch('/api/football-result')
    .then(response => response.text())
    .then(result => setFootballResult(result))
    .catch(err => alert(err))
}

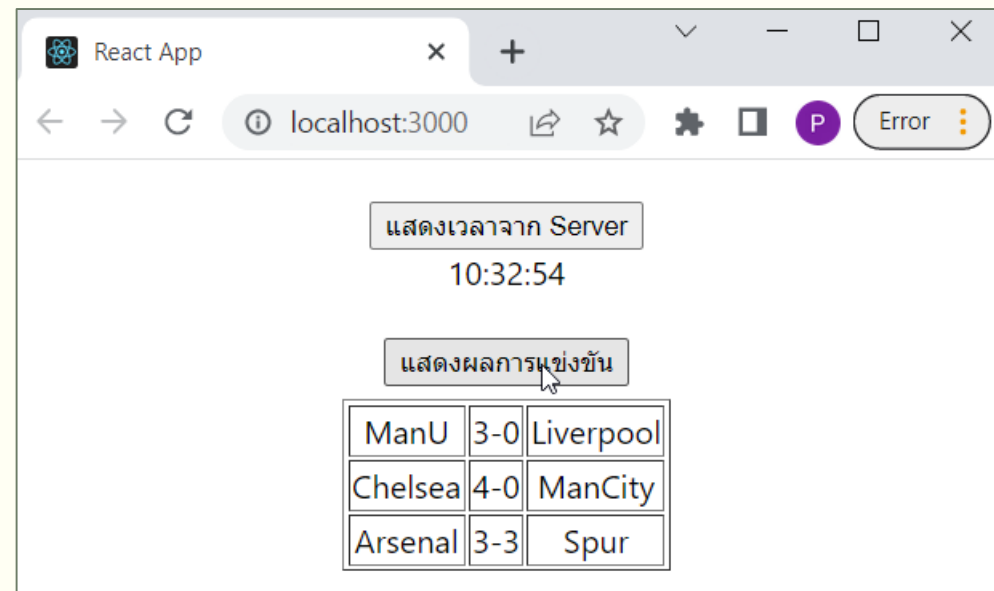
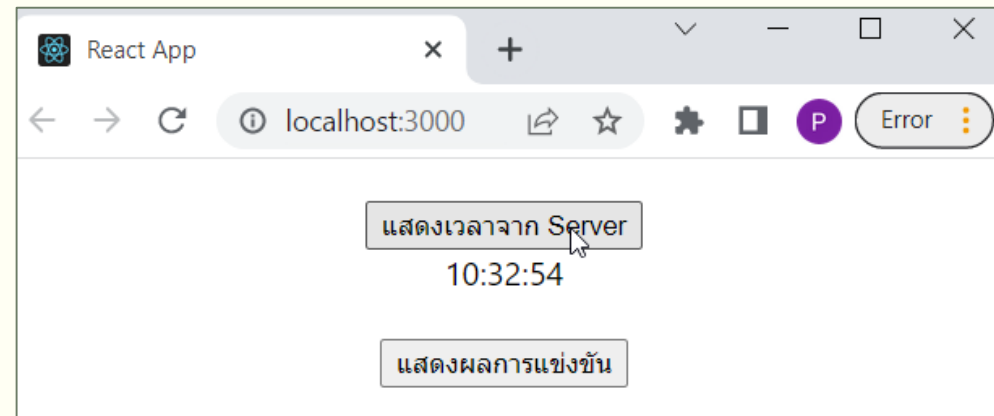
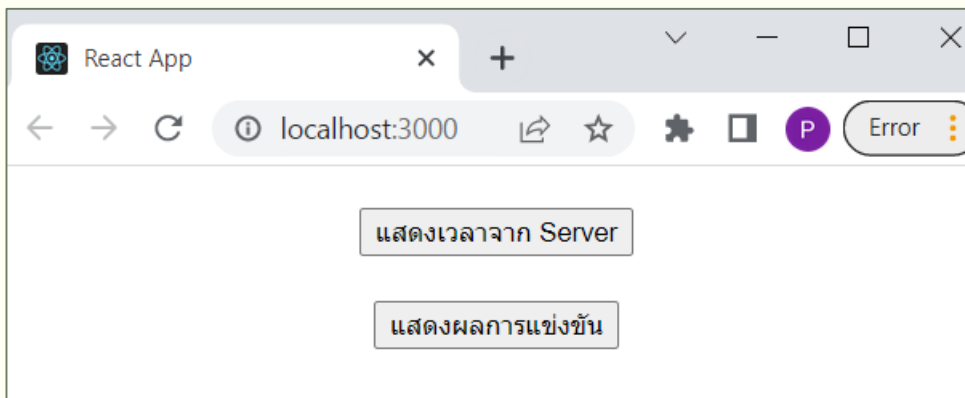
return (
  <div style={{textAlign:'center', marginTop:'20px'}}>
    <button onClick={onClickShowTime}>แสดงเวลาจาก Server</button>
    <div>{serverTime}</div>
    <br/>
    <button onClick={onClickFootballResult}>แสดงผลการแข่งขัน</button><br/>
    <div dangerouslySetInnerHTML={{__html: footballResult}}></div>
  </div>
)
```

การแสดงผลลัพธ์บนคอมพิวเตอร์ทางฝั่ง Local

react/app8-1/local/src/App.js

```
import React from 'react'
import RestBasic from './rest-basic'

export default function App() {
  return <RestBasic/>
}
```



การส่งข้อมูลจากฟอร์มผ่าน REST API

เนื่องจาก React จะติดต่อกับเซิร์ฟเวอร์ผ่าน REST API ดังนั้น เราจึงไม่สามารถส่งข้อมูลจากฟอร์มโดยตรงได้ ซึ่งตามปกติเราจะต้องอ่านข้อมูลจากอินพุตแต่ละตัว แต่ปัจจุบันใน JavaScript มีคลาสที่ชื่อ FormData สำหรับการอ่านข้อมูลทั้งหมดจากฟอร์ม แล้วนำมากำหนดให้แก่ fetch() ก่อนจะส่งไปยังเซิร์ฟเวอร์ จึงช่วยลดความยุ่งยากลงไปได้มาก ส่วนแนวทางการใช้งาน จะแยกพิจารณาตามเมธอดในการส่งข้อมูล ดังนี้

การส่งข้อมูลจากฟอร์มด้วยเมธอด GET

เรามักใช้เมธอด GET กับข้อมูลที่มีจำนวนไม่มาก และไม่เน้นความปลอดภัยในการจัดส่ง ซึ่งหลักการพื้นฐานในส่วนของฟังก์ชัน fetch() ก็เหมือนกับที่เราได้ศึกษามาแล้ว แต่สิ่งที่จะกล่าวเพิ่มเติมก็คือ การอ่านข้อมูลจากฟอร์ม เพื่อนำไปใช้สำหรับการส่งข้อมูลด้วยเมธอด GET โดยมีแนวทางดังต่อไปนี้

- ในส่วนของฟอร์ม เราก็กำหนดอินพุตเพื่อรับข้อมูลไปตามปกติ และต้องสร้าง Reference เพื่อใช้ในการอ้างอิงถึงฟอร์ม
- เมื่อเราจะส่งข้อมูลจากฟอร์ม จะเกิดอีเวนต์ submit ซึ่งเราต้องป้องกันการส่งข้อมูลจากฟอร์มโดยตรงด้วยการเรียกเมธอด event.preventDefault()

การส่งข้อมูลจากฟอร์มผ่าน REST API

- การอ่านข้อมูลทั้งหมดจากฟอร์ม วิธีที่สะดวกคือใช้คลาส FormData ซึ่งมีอยู่แล้วใน JavaScript แต่หากเราต้องการส่งข้อมูลด้วยเมธอด GET จะต้องนำมาแปลงให้เป็นรูปแบบ Query String เช่น การใช้ URLSearchParams เป็นต้น จากนั้นก็นำไปต่อท้าย url เช่น

```
const form = React.useRef()
...
let formData = new FormData(form.current)
let params = new URLSearchParams(formData)
//ผลลัพธ์ที่ได้ เช่น a=123&b=hello&
let url = '/api/test?' + params
```

- ในส่วนของ REST API เราก็ใช้ fetch() ตามหลักการเดิมทั้งหมด

สำหรับแนวทางการนำไปใช้งาน ให้ดูการทดสอบต่อไปนี้จะสร้างฟอร์มที่มีลักษณะดังภาพ แล้วส่งข้อมูลแบบ GET ขึ้นไปยังเซิร์ฟเวอร์ จากนั้นก็แค่ทดสอบอ่านข้อมูลแล้วส่งกลับแบบ JSON และทางฝั่งโลคอลก็นำมาจัดรูปแบบเป็น JSX แล้วค่อยนำไปแสดงผลที่อีลิเมนต์เป้าหมายโดยการอัปเดตค่า State

การส่งข้อมูลจากฟอร์มผ่าน REST API

react/app8-1/local/src/form-get.js

```
import React from 'react'

export default function FormGet() {
  let [searchResult, setSearchResult] = React.useState("")
  const form = React.useRef()

  const onSubmitForm = (event) => {
    event.preventDefault()

    const formData = new FormData(form.current)
    let params = new URLSearchParams(formData) //.toString()
    let URL = '/api/form-get?' + params
```

การส่งข้อมูลจากฟอร์มผ่าน REST API

react/app8-1/local/src/form-get.js (ต่อ)

```
fetch(URL)
.then(response => response.json())
.then(result => {
  let r = (
    <>
    ค้นหา {result.target} ที่ตรงกับ: {result.kw} <br/>
    พบข้อมูลทั้งหมด: {result.results} รายการ
    </>
  )
  setSearchResult(r)
})
.catch(err => alert(err))
}
```

```
return (
  <div style={{textAlign:'center', marginTop:'20px'}}>
    <form ref={form} onSubmit={onSubmitForm}>
      <label>ค้นหา:</label>&nbsp;
      <select id="target" name="target">
        <option value="เว็บ">เว็บ</option>
        <option value="รูปภาพ">รูปภาพ</option>
        <option value="วิดีโอ">วิดีโอ</option>
      </select>&nbsp;
      <input type="text" id="kw" name="kw"/>&nbsp;
      <button>ตกลง</button>
    </form>
    <br/>
    <div>{searchResult}</div>
  </div>
)
```

การส่งข้อมูลจากฟอร์มผ่าน REST API

react/app8-1/server/index.js

```
const express = require('express')
const app = express()
const port = 8000
app.use(express.urlencoded({extended: true}))
app.use(express.json())
app.use(express.static('public'))

app.get('/api/form-get', (request, response) => {
  let t = request.query.target || ''
  let k = request.query.kw || ''
  let n = parseInt((Math.random() * 1000))
```

```
  let r = {
    target: t,
    kw: k,
    results: n
  }
  response.json(r)
})

app.listen(port, () => {
  console.log('Server listening on port ' + port)
})
```

การส่งข้อมูลจากฟอร์มผ่าน REST API

react/app8-1/local/src/App.js

```
import React from 'react'
import FormGet from './form-get'

export default function App() {
  return <FormGet/>
}
```

React App

localhost:3000

ค้นหา: รูปภาพ ▾

React App

localhost:3000

ค้นหา: รูปภาพ ▾

ค้นหา รูปภาพ ที่ตรงกับ: flower
พบข้อมูลทั้งหมด: 591 รายการ

การส่งข้อมูลจากฟอร์มผ่าน REST API

การส่งข้อมูลจากฟอร์มด้วยเมธอด POST

เรามักใช้เมธอด POST กับข้อมูลที่ยาว และ/หรือ ต้องการความปลอดภัยในการจัดส่ง และในการอ่านข้อมูลจากฟอร์มเพื่อส่งไปยังเซิร์ฟเวอร์ผ่าน REST API ก็สามารถใช้ FormData ได้เช่นเดียวกับเมธอด GET แต่ในกรณีนี้ เราต้องนำข้อมูลที่ได้อะมาแปลงเป็น JSON ก่อน แล้วค่อยกำหนดให้แก่ออปชัน body ของฟังก์ชัน fetch() นอกจากนี้ยังต้องกำหนด header อีกด้วย ดังที่เราเคยทำกันมาแล้ว ส่วนขั้นตอนอื่นๆ ก็คล้ายกับเมธอด GET

สำหรับแนวทางการนำไปใช้งาน ให้ดูจากการทดสอบต่อไปนี้ โดยจะสร้างฟอร์มที่มีลักษณะดังภาพ แล้วอ่านข้อมูลด้วย FormData จากนั้นนำมาแปลงเป็น JSON เมื่อส่งขึ้นไปยังเซิร์ฟเวอร์ ก็แค่ทดลองอ่านข้อมูลแล้วจัดรูปแบบเป็นตาราง จากนั้นส่งกลับมายังฝั่งไคลเอนต์ในแบบสตริง แล้วนำไปแสดงผลในอีลิเมนต์ที่จัดเตรียมไว้

การส่งข้อมูลจากฟอร์มผ่าน REST API

react/app8-1/local/src/form-post.js

```
import React from 'react'

export default function FormPost() {
  let [.....] = React.useState("")
  const ..... = React.useRef()

  const onSubmitForm = (event) => {
    event.preventDefault()
    const formData = new FormData(form.current)
    const formEnt = Object.fromEntries(formData.entries())
    fetch(....., {
      method: 'POST',
      body: JSON.stringify(formEnt),
      headers: {'Content-Type': 'application/json'}
    })
  }
}
```

การส่งข้อมูลจากฟอร์มผ่าน REST API

react/app8-1/local/src/form-post.js (ต่อ)

```
.then(response => response.text())
.then(result => setPostedData(result))
.catch(err => alert(err))
}

const inputStyle = {
  margin: '5px 0',
}
```

```
return (
  <div style={{margin:'30px'}}>
    <form ref={form} onSubmit={.....}>
      <div>ติดต่อเรา</div>
      <input type="text" name="name" size="43" placeholder="ชื่อ"
        style={.....} /><br/>
      <input type="email" name="email" size="43" placeholder="อีเมล"
        style={.....}/><br/>
      <textarea name="message" cols="40" rows="4" placeholder="ข้อความ"
        style={.....}></textarea><br/>
      <button>ตกลง</button>
    </form>
    <br/>
    <div .....></div>
  </div>
)
```

การส่งข้อมูลจากฟอร์มผ่าน REST API

react/app8-1/server/index.js

```
const express = require('express')
const app = .....
const port = 8000
app.use(express.urlencoded({extended: true}))
app.use(express.json())
app.use(express.static('public'))

app.post('/api/form-post', (request, response) => {
  let name = request.body.name || ""
  let email = request.body.email || ""
  let msg = .....
```

```
let ..... = `
  <table border="1">
    <caption>ข้อมูลที่ส่งขึ้นไป</caption>
    <tr><td>ชื่อ:</td><td>${.....}</td></tr>
    <tr><td>อีเมล:</td><td>${.....}</td></tr>
    <tr><td>ข้อความ:</td><td>${.....}</td></tr>
  </table>
`

response.send(text)
})

app.listen(port, () => {
  console.log('Server listening on port ' + port)
})
```

การส่งข้อมูลจากฟอร์มผ่าน REST API

react/app8-1/local/src/App.js

```
import React from 'react'
import FormPost from './form-post'

export default function App() {
  return .....
}
```

React App

localhost:3000

ติดต่อเรา

พลกฤษณ์ วงษ์สันติสุข

phollakrit@gmail.com

ต้องการทราบวิธีส่งข้อมูลจากฟอร์มผ่าน REST API ไปยังเซิร์ฟเวอร์ด้วยเมธอด POST

ตกลง

React App

localhost:3000

ติดต่อเรา

พลกฤษณ์ วงษ์สันติสุข

phollakrit@gmail.com

ต้องการทราบวิธีส่งข้อมูลจากฟอร์มผ่าน REST API ไปยังเซิร์ฟเวอร์ด้วยเมธอด POST

ตกลง

ข้อมูลที่ส่งขึ้นไป

ชื่อ:	พลกฤษณ์ วงษ์สันติสุข
อีเมล:	phollakrit@gmail.com
ข้อความ:	ต้องการทราบวิธีส่งข้อมูลจากฟอร์มผ่าน REST API ไปยังเซิร์ฟเวอร์ด้วยเมธอด POST